

第 1 节 实操任务2：裸 API 调用 vs OpenC...

目标：对比实验完成 + 200 字体会文档

2.1 准备工作

确认 Python 3 可用（如何安装Python不赘述了）：

```
1 python3 --version
2 # 需要 3.10+
```

```
huangj2@Levono-V2:~$ python3 --version
Python 3.12.3
```

2.2 裸 API 调用（无状态推理）

创建测试脚本：

```
1 cd ~/opencode-test
```

用任意编辑器创建 `raw_api_test.py`，内容如下：

```
1 """
2 裸 API 调用测试 - 直接调用 DeepSeek API
3 体验「无状态推理」：模型不知道项目背景，不能读写文件
4 """
5 import os
6 import json
7 import urllib.request
8
9 API_KEY = os.environ.get("DEEPSEEK_API_KEY", "")
10 API_URL = "https://api.deepseek.com/chat/completions"
11
12 def call_api(prompt: str) -> str:
13     """直接调用 DeepSeek API"""
14     data = json.dumps({
15         "model": "deepseek-chat",
16         "messages": [{"role": "user", "content": prompt}],
17         "max_tokens": 1000,
18     }).encode("utf-8")
19
20     req = urllib.request.Request(
21         API_URL,
22         data=data,
23         headers={
24             "Content-Type": "application/json",
```

```

25         "Authorization": f"Bearer {API_KEY}",
26     },
27 )
28
29 with urllib.request.urlopen(req) as resp:
30     result = json.loads(resp.read().decode())
31     return result["choices"][0]["message"]["content"]
32
33 if __name__ == "__main__":
34     # 测试：让它分析一个项目
35     print("=" * 60)
36     print("测试：让 AI 分析当前项目的代码结构")
37     print("=" * 60)
38
39     response = call_api(
40         "请分析当前项目的目录结构和代码质量，给出改进建议。"
41     )
42     print(response)
43
44     print("\n" + "=" * 60)
45     print("观察：AI 能看到你的项目文件吗？")
46     print("=" * 60)

```

运行脚本：

```
1 python3 raw_api_test.py
```



```

huangj2@Levono-V2:~/Documents/05-Engineering/.../week1-sdd/project/opencode-test$ python3 raw_api_test.py
测试：让 AI 分析当前项目的代码结构
=====
我将分析您的项目目录结构和代码质量，并提供改进建议。由于我无法直接查看您的项目，请提供以下信息以便我进行针对性分析：

## 需要您提供的信息：
1. **项目类型** (Web应用、移动端、库、CLI工具等)
2. **技术栈** (语言、框架、数据库等)
3. **当前目录结构** (可以粘贴或描述)
4. **代码示例** (关键文件或模块)

## 通用改进建议框架：

### 一、目录结构优化原则
...
推荐结构示例（根据项目类型调整）：
project/
├── src/                # 源代码
│   ├── core/          # 核心业务逻辑
│   ├── api/            # API接口层
│   ├── services/       # 业务服务层
│   ├── models/         # 数据模型
│   ├── utils/          # 工具函数
│   └── config/         # 配置文件
├── tests/              # 测试文件
├── docs/               # 文档
├── scripts/            # 构建/部署脚本
└── package.json        # 依赖管理
...

```

全是幻觉

裸 API 调用的输出——AI 会说「请提供代码」或「我无法访问文件」之类的话

观察要点：

- AI 能看到你的项目文件吗？→ **不能**
- AI 知道你用什么技术栈吗？→ **不知道**
- AI 能执行命令、读写文件吗？→ **不能**

这就是「无状态推理」——模型是一个孤立的函数，输入文本，输出文本，仅此而已。

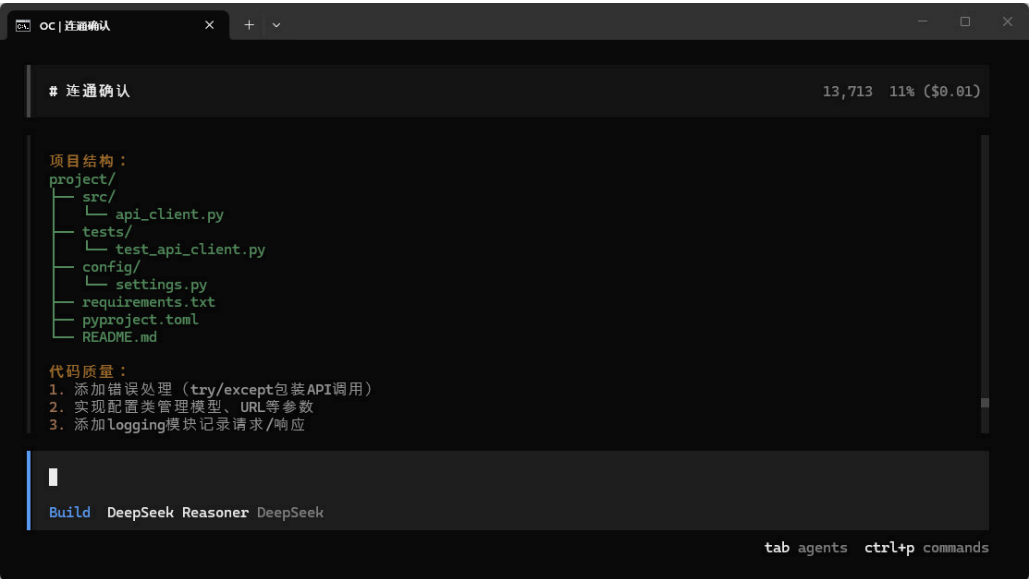
2.3 OpenCode 编排（有状态）

现在用 OpenCode 问完全相同的问题：

```
1 cd ~/opencode-test
2 opencode
```

在 OpenCode 对话框中输入：

```
1 请分析当前项目的目录结构和代码质量，给出改进建议。
```



OpenCode 能分析项目的输出——它会读取文件、列出目录结构、给出具体建议。

观察要点：

- AI 能看到你的项目文件吗？→ 能（它会用 Glob/Read 工具扫描目录）

- AI 知道文件内容吗？→ **知道**（它会读取文件内容）
- AI 能给出具体建议吗？→ **能**（基于实际文件内容）

这就是「有状态编排」——同一个模型，加了编排器之后，它能读文件、能搜索、能分析，变成了一个有感知能力的系统。

2.4 写对比体会

创建文件 `comparison.md`，写 200 字的对比体会。参考模板：

```
1 # 裸 API vs OpenCode 对比体会
2
3 ## 实验过程
4 - 裸 API 调用：[描述你观察到的现象]
5 - OpenCode 编排：[描述你观察到的现象]
6
7 ## 关键差异
8 - [差异 1：是否能读文件]
9 - [差异 2：是否有上下文感知]
10 - [差异 3：建议是否具体]
11
12 ## 我对「无状态推理」和「有状态编排」的理解
13 [用自己的话总结]
```

作业：请提交你写的对比体会文档

核心收获

	裸 API 调用	OpenCode 编排
本质	无状态推理	有状态编排
能否看文件	✗	✓
能否执行命令	✗	✓
上下文感知	✗	✓
类比	打电话问路	开着导航走

关键洞察：模型是同一个模型（DeepSeek），差别在于编排器赋予了它“感知”和“行动”的能力。

完成！ 你已亲手体验了无状态推理和有状态编排的差异。